WHAM-XREX-PFMG474

Pin Mapping Reference

September 2026

Rev	Date	Author	Description
A	2026-09-18	–	Initial issue. Covers all four Transrex channels' fault/feedback/drive/enable signals, system-wide interlock pins, the permanently-retired E-stop pin, and the fiber-optic cross-board wiring for the Transrex simulator.

1 Purpose and Scope

This document is the curated, organized reference for every pin this firmware actually assigns a role to – derived from, and cross-checked against, `docs/pin_mapping_v4.csv` (the authoritative raw pin data) and the current firmware source under `src/` (chiefly `ctrlr_config.h`, `xrex_io.c`, `state_machine.c` and `board_io.c`, which holds the pin table and GPIO setup). Where this document and the firmware source ever disagree, the firmware source is correct – this is a reference for humans, not a second source of truth a build depends on.

Scope: pin identity, direction, detection/drive mechanism, fault polarity, and the SCPI commands that read or write each signal. Board bring-up, power-stage safety procedure, and build/flash mechanics are covered by `docs/sop/wham_xrex_pfmg474_sop.tex` and `docs/serial_reflash_guide.md`, not repeated here.

2 The XREX Per-Channel Naming Scheme

`pin_mapping_v4.csv`'s "XREX Pin Name" column names each of the four Transrex channels' own signals `XR`n`_*`, where $n \in \{1, 2, 3, 4\}$ maps 1:1 to this firmware's existing WHAM/HRTIM channel numbering (confirmed via `XR1_DRIVE` = `PA8` = `HRTIM1_CHA1`, matching the Phase U assignment already established before the XREX naming existed). Eight signal categories exist per channel:

Category	Meaning
_WATER_FLT	Water fault input (General Fault)
_TMP_FLT	Temperature fault input (General Fault)
_ENERPRO_FLT	Enerpro fault input (General Fault)
_OCP	Overcurrent-protect fault input (its own fault type)
_FEEDBACK	Current-feedback input (timer capture; a pure relabel of the pre-existing PFM_Input_0n pins, no functional change)
_DRIVE	PFM/HRTIM output (a pure relabel of the pre-existing HRTIM phase-output pins, no functional change)
_ENA_OUT	Channel enable fiber output
_CONTACT_OUT	Contactor fiber output

3 Per-Channel Pin Table

Signal	V4 Name	Pin	Dir	Mechanism
XR1_WATER_FLT	GateDriverStatus_01	PE0	GPI	EXTI
XR2_WATER_FLT	GateDriverStatus_02	PE1	GPI	EXTI
XR3_WATER_FLT	GateDriverStatus_03	PE2	GPI	EXTI
XR4_WATER_FLT	GateDriverStatus_04	PE3	GPI	EXTI
XR1_TMP_FLT	GateDriverStatus_05	PE4	GPI	EXTI
XR2_TMP_FLT	GateDriverStatus_06	PE5	GPI	EXTI
XR3_TMP_FLT	GateDriverStatus_07	PE6	GPI	EXTI
XR4_TMP_FLT	GateDriverStatus_08	PE7	GPI	EXTI
XR1_ENERPRO_FLT	GateDriverStatus_09	PE8	GPI	EXTI
XR2_ENERPRO_FLT	GateDriverStatus_10	PE9	GPI	EXTI
XR3_ENERPRO_FLT	GateDriverStatus_11	PE10	GPI	EXTI
XR4_ENERPRO_FLT	GateDriverStatus_12	PE11	GPI	EXTI
XR1_OCP	GPInput_03	PF4	GPI	Polled
XR2_OCP	GPInput_07	PF8	GPI	Polled
XR3_OCP	GPInput_11	PF12	GPI	Polled
XR4_OCP	GPInput_04	PF5	GPI	Polled

Signal	V4 Name	Pin	Dir	Mechanism
XR1_FEEDBACK	PFM_Input_01	PA15	GPI	TIM2_CH1 capture
XR2_FEEDBACK	PFM_Input_02	PD4	GPI	TIM2_CH2 capture
XR3_FEEDBACK	PFM_Input_03	PB2	GPI	TIM5_CH1 capture
XR4_FEEDBACK	PFM_Input_04	PC12	GPI	TIM5_CH2 capture
XR1_DRIVE	PHASE_U_PIN	PA8	AF13	HRTIM1_CHA1
XR2_DRIVE	PHASE_V_PIN	PA10	AF13	HRTIM1_CHB1
XR3_DRIVE	PHASE_W_PIN	PB12	AF13	HRTIM1_CHC1
XR4_DRIVE	PHASE_X_PIN	PB14	AF13	HRTIM1_CHD1
XR1_ENA_OUT	GPOut_01	PG0	GPO	Push-pull
XR2_ENA_OUT	GPOut_02	PG1	GPO	Push-pull
XR3_ENA_OUT	GPOut_03	PG2	GPO	Push-pull
XR4_ENA_OUT	GPOut_04	PG3	GPO	Push-pull
XR1_CONTACT_OUT	GPOut_05	PG4	GPO	Push-pull
XR2_CONTACT_OUT	GPOut_06	PG5	GPO	Push-pull
XR3_CONTACT_OUT	GPOut_07	PG6	GPO	Push-pull
XR4_CONTACT_OUT	GPOut_08	PG7	GPO	Push-pull

"EXTI" = interrupt-driven via the shared GateDriverStatus EXTI setup, delegating its fault decision to `XrexIo_EvaluateGateDriverFault()` (`xrex_io.c`). "Polled" = read every `PID_Update()` tick and once at boot by `XrexIo_PollOcpFaults()` – not EXTI-driven despite matching Water/Temp/Enerpro's own category, because 3 of the 4 OCP pins' EXTI line numbers directly conflict with the already-claimed GateDriverStatus EXTI lines (STM32's 16 EXTI lines are shared project-wide, one GPIO port per line via `SYSCFG_EXTICR`).

4 Fault Detection: Gating and Polarity

Per-channel gating (the core architectural rule): a channel's own `_WATER_FLT`/`_TMP_FLT`/`_ENERPRO_FLT`/`_OCP` pins only count toward a fault when that channel is currently enabled (`SOURce:ENABle?`) – confirmed on real hardware: with only channel 3 enabled, an otherwise-faulted board correctly and exclusively reported `STATE?` → `OK FAULT OVERCURRENT 3`.

Polarity – every category above is independently configurable (`ctrlr_config.h`), currently all four defaulting to `FAULT_POLARITY_NORMALLY_HIGH` (pins read HIGH in healthy operation; a LOW reading is a fault):

Constant	Current value	Governs
XR_WATER_FLT_POLARITY	NORMALLY_HIGH	Water fault, all 4 ch.
XR_TMP_FLT_POLARITY	NORMALLY_HIGH	Temp fault, all 4 ch.
XR_ENERPRO_FLT_POLARITY	NORMALLY_HIGH	Enerpro fault, all 4 ch.
XR_OCP_FLT_POLARITY	NORMALLY_HIGH	OCP fault, all 4 ch.

CAUTION: This NORMALLY_HIGH default is a *deliberate override* of a value that was itself confirmed against real hardware on 2026-09-08 (a GDS? snapshot of these exact 12 physical pins found 11 of 12 reading LOW, consistent with LOW being the healthy state at the time). The current NORMALLY_HIGH default was confirmed directly by the project owner and has *not yet been independently re-verified against a real, non-floating healthy signal* – every real-hardware check so far exercised only the floating/unconnected state (GPIO_PULLDOWN), which reads as faulted under either polarity and cannot itself validate which polarity is physically correct. Re-verify once real Water/Temp/ Enerpro/OCP signal sources are connected.

5 System-Wide (Non-Per-Channel) Pins

V4 Name	Pin	Dir	Role
GPInput_12	PF13	GPI	External-enable interlock. ARM/SHOT:STARt refuse unless HIGH; continuously monitored while ARMED/FIRING. Commands: EXTernal:ENAble, EXTernal:ENAble?, EXTernal:INPut?. GPIO_PULLDOWN fail-safe (floating = LOW = no permission). Moved here 2026-09-17 from PF15.
Fiber_Enable	PF15	GPI	External trigger. A rising edge while ARMED fires a shot (calls the same SM_Fire() SHOT:STARt uses). Independent of the interlock above as of 2026-09-17 – no longer requires EXTernal:ENAble on first. Commands: EXTernal:TRIGger, EXTernal:TRIGger?, EXTernal:TRIGger:INPut?. GPIO_PULLDOWN.
GPOut_Enable_Pin	PC13	GPO	Named board output, default HIGH at boot (GPOUT_ENABLE_DEFAULT_HIGH, ctrlr_config.h). Commands: GPOut:ENAble, GPOut:ENAble?.

V4 Name	Pin	Dir	Role
PWM_Alt_Enable	PC15	GPO	Named board output, default HIGH at boot (PWMALT_ENABLE_DEFAULT_HIGH, ctrlr_config.h). Commands: PWMAlt:ENAble, PWMAlt:ENAble?.
GPOut_11	PD0	GPO	Generic diagnostic output, default LOW at boot. Commands: DIAGnostic:GPOut11, DIAGnostic:GPOut11?.
GPOut_12	PD1	GPO	Generic diagnostic output, default LOW at boot. Commands: DIAGnostic:GPOut12, DIAGnostic:GPOut12?.
UART_TX_LED	PD2	GPO	Front-panel LED, active high: lit while the controller transmits on any UART (30 ms after the last byte; switched off by a timer and DMA, led_indicators.c). Added 2026-10-07; V3 GPOut_03.
UART_RX_LED	PD3	GPO	Front-panel LED: lit while the controller receives. Added 2026-10-07; V3 GPOut_04.
STATE_IDLE_LED	PF14	GPO	Front-panel LED, lit if and only if the state machine is IDLE. Added 2026-10-07.
STATE_ARMED_LED	PF2	GPO	Lit if and only if ARMED. Added 2026-10-07; V3 GPInput_01.
STATE_FIRING_LED	PC3	GPO	Lit if and only if FIRING. Added 2026-10-07.
STATE_FAULT_LED	PC2	GPO	Lit if and only if FAULT. Added 2026-10-07. DIAGnostic:LEDs? reads all six back.
STM_Enable_Pin	PC14	GPO	**Considered and rejected** for the external-enable role (2026-09-16) – documented direction is OUTWARD (the MCU drives it), the wrong direction for something meant to be read. Not otherwise used.
BOOT0	PB8	N/A	**Not available for GPIO reuse** – confirmed via a live FLASH_OPTR register read (DIAGnostic:OPTBytes?) that nSWBOOT0=1 on this chip, meaning BOOT0 is genuinely pin-sampled on every reset, not just first power-on.

6 Retired Pins – Do Not Reuse

CAUTION: PG10 must never be configured as a GPIO peripheral pin. It is electrically the MCU's own NRST pin, tied to the same net as the ST-Link/Molex debug connector – confirmed directly against the board schematic on 2026-09-17, after a hardware investigation (a new DIAGnostic:RSTCause? diagnostic, reading the real RCC->CSR reset-cause flags, proved a genuine PINRSTF-flagged hardware reset, not a firmware bug) found that driving a signal into this pin resets the MCU. The reset persisted even after removing this firmware's own GPIO configuration for the pin entirely, proving it is a board-level electrical fact that cannot be routed around in software. docs/pin_mapping_v4.csv's own "NRST" label for this net, once believed stale/incorrect, was correct all along.

This pin was originally used for an emergency-stop feature (SM_FAULT_EMERGENCY_STOP, EMERGency:ENAble and friends). That software remains in the firmware, permanently dormant (EMERGency:ENAble defaults off, and reads no pin at all while off) – any future emergency-stop implementation must use a genuinely different, schematic-confirmed-free pin.

7 Transrex Simulator: Fiber-Optic Cross-Board Wiring

A second, physically identical board ("the simulator," BUILD_TARGET_SIMULATOR) plays the role of a Transrex for full-stack bench testing without driving real power hardware. Every connection between the two boards is a fiber-optic point-to-point link, not a plain electrical wire – each pin position has either an LED transmitter or a photodiode receiver permanently fixed to it, identical on both boards, so a signal must go from a transmitter pin to a receiver pin over its own fiber, regardless of pin number.

7.1 DRIVE / FEEDBACK (unaffected by the fiber pool below)

From	To
Controller XR*n*_DRIVE (HRTIM out)	Simulator XR*n*_FEEDBACK (TIM capture in)
Simulator XR*n*_DRIVE (HRTIM out)	Controller XR*n*_FEEDBACK (TIM capture in)

7.2 Fiber Transceiver Pool and Capacity Constraint

The simulator has 12 transmitters (GPOut_01–GPOut_12) and 18 receivers (GateDriverStatus_01–_12 plus GPInput_03/04/07/08/11/12) available for this purpose (PFM_Input/HRTIM pins excluded – already assigned above). The controller needs **16** fault-signal receivers driven (12 GateDriverStatus for Water/Temp/Enerpro, 4 GPInput for OCP), but the simulator has only 12 transmitters – a shortfall of exactly 4.

Resolved with four physical 1-to-2 fiber splitter cables: each channel's Water and Temp faults share one simulator transmitter, split to both controller receivers. This means Water and Temp can no longer be independently faulted per channel, only together – freeing exactly enough transmitter budget (4 combined Water+Temp, 4 Enerpro, 4 OCP = 12) to cover OCP as well.

Sim. TX	Splits to (controller RX)	Signal
GPOut_01	GateDriverStatus_01 + _05 (PE0+PE4)	XR1 Water+Temp
GPOut_02	GateDriverStatus_02 + _06 (PE1+PE5)	XR2 Water+Temp
GPOut_03	GateDriverStatus_03 + _07 (PE2+PE6)	XR3 Water+Temp
GPOut_04	GateDriverStatus_04 + _08 (PE3+PE7)	XR4 Water+Temp
GPOut_05	GateDriverStatus_09 (PE8)	XR1 Enerpro
GPOut_06	GateDriverStatus_10 (PE9)	XR2 Enerpro
GPOut_07	GateDriverStatus_11 (PE10)	XR3 Enerpro
GPOut_08	GateDriverStatus_12 (PE11)	XR4 Enerpro
GPOut_09	GPInput_03 (PF4)	XR1 OCP
GPOut_10	GPInput_07 (PF8)	XR2 OCP
GPOut_11	GPInput_11 (PF12)	XR3 OCP
GPOut_12	GPInput_04 (PF5)	XR4 OCP

OCP assignment matches the channel order already built into the controller's own pin table (`board_io.c`) – XR1/XR2/XR3/XR4 → PF4/PF8/PF12/PF5, an irregular but pre-existing order.

7.3 ENA_OUT / CONTACT_OUT Reception

The controller's existing, already-built GPOut_01 through GPOut_08 transmitters (XR1_ENA_OUT through XR4_CONTACT_OUT) are received on the simulator's own GateDriverStatus_01 through GateDriverStatus_08 position pins (PE0–PE7), matching number to the controller's own GPOut_01 through GPOut_08. This leaves the simulator's GateDriverStatus_09 through GateDriverStatus_12 position pins, and all 6 GPInput pins (_03/_04/_07/_08/_11/_12), spare.

External-enable (PF13) and external-trigger (PF15) are **out of scope** for the simulator – those represent an operator/facility permissive and a manual trigger, not something a real Transrex itself drives. They continue to be tested via the existing DIAGnostic:GPOut11/GPOut12 loopback method.

8 References

- `docs/pin_mapping_v4.csv` – the authoritative raw pin data this document is derived from.
- `src/config/ctrlr_config.h` – polarity constants, board identity, default levels.
- `src/app/protection/xrex_io.c` and `.h` – per-channel fault-pin interpretation.
- `src/bsp/stm32g4/board_io.c` – the pin table behind every named board signal (including the OCP inputs) and their GPIO configuration.
- `src/app/control/state_machine.c` and `.h` – fault-type architecture: `SM_FAULT_GENERAL`, `SM_FAULT_OVERCURRENT`, `SM_FAULT_EXTERNAL_ENABLE`, `SM_FAULT_EMERGENCY_STOP`, `SM_FAULT_ENABLE_OUTPUT`.
- `docs/command_reference.md` – full SCPI command protocol for every command named in this document.
- `docs/changelog.txt` – dated development history and real-hardware verification numbers for everything in this document.
- The Transrex simulator implementation plan (session-local, not part of this repository) – Section 7 is a snapshot of that plan's wiring section as of this revision.